Trabajo Práctico Nº 3

Sección crítica - Sincronización

1. En este ejercicio vamos a tener 3 clases: una clase `Main`, una clase `CuentaBanco` y una clase `VerificarCuenta` (Hilo).

a) Ejecute el código. Comente el resultado de la ejecución. ¿Qué podría ocurrir con el balance de la cuenta si no se protege el acceso?

b) ¿Qué corrección debería realizar para mejorar la protección de los datos?

Main.java

```java
public class Main {
    public static void main(String[] args) {
        CuentaBanco cb = new CuentaBanco(50);
        VerificarCuenta vcL = new VerificarCuenta(cb);
        VerificarCuenta vcJ = new VerificarCuenta(cb);
        Thread lucas = new Thread(vcL, "Lucas");
        Thread jere = new Thread(vcJ, "Jere");
        lucas.start();
        jere.start();
    }
}
```

CuentaBanco.java

```java
public class CuentaBanco {
    private int balance;

    public CuentaBanco(int balance) {
        this.balance = balance;
    }

    public int getBalance() {
        return this.balance;
    }

    public void retiroBancario(int retiro) {
        this.balance -= retiro;
    }
}
```

VerificarCuenta.java

```java
public class VerificarCuenta implements Runnable {
    private CuentaBanco cb;

    public VerificarCuenta(CuentaBanco unaCuenta) {
        this.cb = unaCuenta;
    }

```

```
    private void hacerRetiro(int cantidad)
            throws InterruptedException {
        if (cb.getBalance() >= cantidad) {
            System.out.println (Thread.currentThread().getName() +
                " está realizando un retiro de: " + cantidad);

            Thread.sleep(1000);
            cb.retiroBancario(cantidad);
            System.out.println(Thread.currentThread().getName() +
                ": Retiro realizado");

            System.out.println(Thread.currentThread().getName() +
                ": Los fondos son de: " + cb.getBalance());
        } else {
            System.out.println("No hay suficiente dinero en la cuenta"
                + " para realizar el retiro Sr. "
                + Thread.currentThread().getName());

            System.out.println("Su saldo actual es de: "
                + cb.getBalance());
            Thread.sleep(1000);
        }
    }

    public void run() {
        for (int i = 0; i <= 3; i++) {
            try {
                this.hacerRetiro(10);
                if(cb.getBalance() < 0) {
                    System.out.println("Cuenta está sobregirada");
                }
            } catch (InterruptedException ex) {
                Logger.getLogger(VerificarCuenta.class.getName()).
                    log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

c) Modificar el código anterior de la siguiente manera. Ejecutar nuevamente y comentar el resultado ¿Cuál es la diferencia en ambos casos? ¿Cuál le parece que sea una alternativa mejor? ¿Qué corrección debería realizar para mejorar la protección de los datos?

Main.java (modificado)

```
public class Main {
    public static void main(String[] args) {
        CuentaBanco cb = new CuentaBanco(50);
        VerificarCuenta vcL = new VerificarCuenta(cb);
        VerificarCuenta vcJ = new VerificarCuenta(cb);
        Thread lucas = new Thread(vcL, "Lucas");
        Thread jere = new Thread(vcJ, "Jere");
        lucas.start();
        jere.start();
    }
```

```
}
```

CuentaBanco.java (modificado)

```
public class CuentaBanco {
    private int balance;

    public CuentaBanco(int balance) {
        this.balance = balance;
    }

    public int getBalance() {
        return this.balance;
    }

    public boolean hacerRetiro(int cantidad)
            throws InterruptedException {
        boolean puedeRetirar = this.balance >= cantidad;
        if (puedeRetirar) {
            System.out.println ( Thread.currentThread().getName() +
                " está realizando un retiro de: " + cantidad);

            Thread.sleep(1000);
            this.balance -= cantidad;
            System.out.println(Thread.currentThread().getName() +
                ": Retiro realizado");

            System.out.println(Thread.currentThread().getName() +
                ": Los fondos son de: " + this.balance);
        } else {
            System.out.println("No hay suficiente dinero en la cuenta"
                + " para realizar el retiro Sr. "
                + Thread.currentThread().getName());

            System.out.println("Su saldo actual es de: "
                + this.balance);
            Thread.sleep(1000);
        }
        return puedeRetirar;
    }
}
```

VerificarCuenta.java (modificado)

```
public class VerificarCuenta implements Runnable {
    private CuentaBanco cb;

    public VerificarCuenta(CuentaBanco unaCuenta) {
        this.cb = unaCuenta;
    }

    public void run() {
        for (int i = 0; i <= 3; i++) {
            try {
                if(!cb.hacerRetiro(10)) {
```

```
                    System.out.println("Cuenta está sobregirada");
                }
            } catch (InterruptedException ex) {
                Logger.getLogger(VerificarCuenta.class.getName()).
                    log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

2. Realice un programa en el que se presente la interacción entre dos entidades que afectan un recurso compartido, al que llamaremos "Energía", inicializado en 10 unidades. La dinámica es la siguiente:

- La Criatura Oscura (hilo 1) nos drena (-) 3 unidades de Energía.
- El Sanador (hilo 2) nos revitaliza (+) 3 unidades de Energía.

a) Asegúrese de considerar las operaciones de adquisición de energía, su modificación y su posterior actualización en el recurso compartido. Ejecuta el programa varias veces y registra el valor resultante en cada ocasión.

b) Con base en sus observaciones, ¿qué conclusiones puede extraer?

c) Luego, proceda a modificar el programa utilizando una herramienta para sincronizar la ejecución de los hilos.

3. En una tienda de mascotas están teniendo problemas para tener a todos sus hámster felices. Los hámster comparten una jaula en la que hay un plato con comida, una rueda para hacer ejercicio, y una hamaca en la que pueden descansar. Todos los hamsters quieren comer del plato, correr en la rueda y luego descansar en la hamaca (no necesariamente en ese orden). Pero se encuentran con el inconveniente de que solo uno de ellos puede comer del plato, solo uno puede correr en la rueda y solo uno puede descansar en la hamaca.

a) Implemente un programa para simular la situación planteada, en donde todos los hámster puedan realizar todas las actividades.

Nota: considere que todas las actividades consumen cierto tiempo, por lo que para la simulación se sugiere asignar ese tiempo con el método "`sleep()`".

4. Desarrolle un programa que simule el trabajo en una obra de construcción donde intervienen cuatro trabajadores: dos albañiles, un gasista y un electricista. Todos ellos deben coordinar utilizando solo una única caja de herramientas compartida para tomar y devolver las herramientas que necesitan para realizar sus labores. Esta caja dispone en total de cinco herramientas: una cuchara de albañil, un buscapolos, una llave de caño, un nivel láser y una cinta métrica.

Para realizar sus tareas, cada albañil toma una herramienta cada vez, alternando según lo

que requiera entre la cuchara de albañil, el nivel de burbuja o la cinta métrica. Por su parte, el electricista utilizará el buscapolos o la cinta métrica, mientras que el gasista usará la llave de caño o el nivel de burbuja. El programa deberá gestionar de forma correcta para que ninguna herramienta sea utilizada por más de un operario de manera simultánea.

5. Se dispone de un arreglo de números enteros comprendidos entre 1 y 10 con capacidad para 50.000 números. Se desea disponer de un programa que efectúe la suma de los componentes del arreglo utilizando una estrategia concurrente, que obtenga la suma final mediante sumas parciales calculadas por una serie de hilos independientes.

Escriba un programa que realice la tarea propuesta, y que cumpla con las especificaciones siguientes:

a) El arreglo debe llenarse con números enteros entre 1 y 10 generados aleatoriamente. Nota: utilizar la clase `Random` de java.

b) Habrá k hilos, entre los cuales se dividirá el trabajo a realizar: cada uno debe ocuparse de proporcionar la suma parcial de un segmento del arreglo.

c) Habrá un programa principal que creará y rellenará el arreglo de datos, y escribirá el valor final resultante de la suma realizada por los hilos.

d) El arreglo de números y la suma final **deben** vivir dentro de un recurso compartido independiente a los hilos.

6. En un sistema de reserva de espacios en un parque temático en línea, los visitantes pueden seleccionar y reservar áreas específicas para actividades y entretenimiento. Cada área tiene un número limitado de espacios disponibles.

Desarrolle un programa que permita a los visitantes reservar áreas según la disponibilidad. Cuando un visitante elige una ubicación, el sistema verifica si está libre y, si es así, lo reserva y emite un comprobante. Es esencial evitar reservas duplicadas en la misma área. El sistema debe proporcionar una experiencia fluida para que los visitantes elijan libremente entre las áreas disponibles mientras se asegura que no se realicen reservas dobles.

7. Implemente una clase `Auto` con sus atributos representativos (patente, modelo, marca, km_recorridos, etc). Un auto al recorrer varios kilómetros consume combustible, y cuando llega al nivel de la reserva debe cargar combustible para poder continuar. Considere que se desea simular la siguiente situación: Varios autos recorriendo la ciudad y un surtidor que despacha combustible.

- Realice un programa para representar la situación. ¿Cómo representará los autos? ¿Y el surtidor? Realice una prueba del programa con 5 autos como mínimo.
- Considere que el surtidor tiene capacidad para un máximo de litros de combustible, que se van gastando a medida que los autos pasan a abastecerse.
- Cada vez que se realiza una carga de combustible escriba un mensaje apropiado para

informar el estado actual del surtidor.

8. Diseñar un programa en Java para la gestión de impresión de letras A, B y C. Para ello, utilizaremos 3 hilos, pero se quiere limitar la posible salida de caracteres por pantalla para que no se produzca de cualquier forma sino con la secuencia predefinida: ABBCCC, por ejemplo: ABBCCCABBCCC...

Se recomienda el uso de una variable compartida por los hilos que sirva para asignar turnos, de manera que cada hilo imprima su letra cuando sea su turno. De esta forma, cuando se crean los hilos, a cada uno se le asigna un identificador para que puedan comprobar cuando es su turno y que mientras no sea su turno, no haga nada.

A tener en cuenta: Cada hilo sabe la cantidad de veces que debe imprimir. Por ejemplo: El `hiloB` sabe que tiene que imprimir 2 veces la letra B. Podría ocurrir que la cantidad de veces que se tiene que escribir cada letra varíe (por ejemplo, 4 veces A, 2 veces B y 5 veces C), pero siempre se debe respetar el orden en que se imprimen las letras, es decir primero A, luego B y después C.

9. Retome el ejercicio 7, de este trabajo práctico.

Considere ahora la existencia de un Camión que se encarga de recargar el surtidor al máximo definido. El camión pasa cada cierto tiempo, y una vez que llegue todos los autos deben hacerse a un lado para dejar que el camión realice la carga. Una vez que finalice, los autos pueden volver a hacer la fila para cargar combustible.

a) ¿Qué dificultad encontró para implementar esta nueva funcionalidad?